

Two Integer Sum II

Solved

Given an array of integers numbers that is sorted in **non-decreasing order**.

Return the indices (**1-indexed**) of two numbers, [index1, index2], such that they add up to a given target number target and index1 < index2. Note that index1 and index2 cannot be equal, therefore you may not use the same element twice.

There will always be **exactly one valid solution**.

Your solution must use $O(1)$ additional space.

Example 1:

Input: numbers = [1,2,3,4], target = 3

Output: [1,2]

Explanation:

The sum of 1 and 2 is 3. Since we are assuming a 1-indexed array, index1 = 1, index2 = 2. We return [1, 2].

Constraints:

- $2 \leq \text{numbers.length} \leq 1000$
- $-1000 \leq \text{numbers}[i] \leq 1000$
- $-1000 \leq \text{target} \leq 1000$

```
class Solution:
```

```
    def twoSum(self, numbers: List[int], target: int) -> List[int]:
```

```
        if not numbers:
```

```
            return []
```

```
        t = dict()
```

```
        for i, v in enumerate(numbers):
```

```
            t[v] = i
```

```
        print(t)
```

```
        for i, v in enumerate(numbers):
```

```
            if target - v in t.keys():
```

```
                return [min(i,t[target - v])+1, max([i,t[target - v]])+1]
```

```
        return []
```